

Document 07 — AI Agents → Project 2

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

[06_tools_function_calling](#)

How to Read & Practice This Document

- **What:** the loop that turns tool-calling into behavior with many steps, on its own.
- **Why:** "agent" stops being just a buzzword once you've built this loop yourself, by hand, once — before ever using a library that does it for you.
- **When:** for tasks that need several steps, decided by the model, one after another — not simple one-shot Q&A, which doesn't need a loop at all.
- **How to practice:** 1. Read the material once, especially the ReAct paper's example. Just get the shape of it. 2. Trace a loop by hand on paper (**Basic**) before writing any code — this step is not optional. 3. Build the loop yourself (**Intermediate**) before using `create_agent` — you need to build this basic piece once, yourself. 4. Try **Project 2** without looking at any old solution. Let yourself get stuck before asking for a hint. 5. Use **Hint 1 or Hint 2** only after really trying. Ask for the full solution only if you say "**Show me the solution.**" 6. Before moving on, explain out loud why every agent loop needs a hard limit on steps. If you can't, you're not done.

Jump to: [Core Concepts](#) · [Practice Exercises](#) · [Build Task](#)

The Story — what this document is actually building

Picture this: Doc06 gave the model exactly one shot per turn — call a tool, get a result, done, conversation over. Real tasks are rarely that tidy: they take several steps, and which step comes next depends on what the last one actually returned. This document is where a single tool call turns into a loop that keeps deciding its own next step, on its own.

First, the ReAct loop is really just Doc06's one round, repeated: ask the model what to do next (think), run whatever it picked (act), feed the real result back in (observe) — nothing new gets invented here, the "reasoning" is just the model seeing each step's result before deciding the next one. **Second**, every thought, action, and result gets appended to a growing scratchpad, the loop's own message history — the model's next decision is only ever as good as what actually survives in

that history, so a dropped or badly-written result makes the model effectively "forget" it happened. **Third**, the loop needs a hard stop that lives in *your* code, not the model's judgment — a confusing tool result, an unclear task, or a slightly broken tool can make the model ask for the same action over and over, and only a limit you enforce yourself, from the outside, actually guarantees it stops. **Fourth**, once a loop is running many steps instead of one, the same handful of failures show up again and again: believable-but-wrong tool arguments, a tool error the model quietly ignores and reports as a fake success, and the infinite loop the step limit exists to prevent in the first place.

That's the whole story: one repeated round, a scratchpad to remember it by, and a limit to stop it — three pieces, one loop. The Build Task at the end asks you to wire your Doc06 tool library into exactly this loop, because Project 2 *is* that loop doing real, multi-step work, using tools you already built.

Core Concepts (read this first — everything you need is here)

The ReAct loop: think → act → observe

An agent, with the fancy language stripped away, is just a loop: ask the model what to do next (**think**), run the tool it picked (**act**), feed the result back in (**observe**), repeat — until the model decides it has enough to just answer directly instead of calling another tool. **Why this simple loop is the whole foundation of "agent" behavior:** many-step reasoning comes just from repeating single tool calls (Doc06) and letting the model see each result before deciding the next step — there's no separate hidden "planning brain," the plan builds up one call at a time. **How it's different from a single tool call:** Doc06 was one round — the model calls a tool, you return the result, done. An agent loop repeats that round, feeding each result back as new context, until the model stops asking for tools and gives a final answer.

The scratchpad: how the loop remembers its own steps

As the loop runs, every thought, action, and result gets added to the growing message history (sometimes called the "scratchpad") — the exact same idea as Doc03's "no built-in memory," just happening *within* one task instead of across a whole conversation. **Why this matters:** the model's next decision is only as good as what it can actually see in that history — if a tool's result gets dropped or written badly before being fed back in, the model effectively "forgets" it happened, and might repeat the same action.

Setting a limit on the number of steps

The loop needs a way to know it's done: either the model gives a final answer with no more tool calls, or an outside limit is hit. **Why the limit isn't optional:** the model deciding "I should call one more tool" is itself just a guess, not a guarantee — a confusing tool result, an unclear task, or a slightly broken tool can make the model ask for the same (or a similar) action over and over, forever. Without a hard limit set in *your* code (not just hoped for from the model), one bad run can loop

forever, wasting time and — with paid APIs — real money. **How a good limit works:** count the number of steps yourself, outside the model's control, and when it's reached, raise a clear, named error carrying the partial history with it — so the failure is easy to understand, not just "it hung."

Common single-agent failures, worth naming now

Wrong tool arguments — the model calls a tool with believable-looking but wrong values (checked at Doc06's shape level, but the actual *values* can still be logically wrong). **Ignoring a failure** — a tool returns an error, but the model carries on as if it worked, stating a made-up result as fact; this is exactly why Doc06's advice to return clear errors matters — it gives the model a real chance to react correctly, though it doesn't guarantee it will. **Infinite loop** — covered above. All three of these are exactly what [14_debugging_lab](#) will have you practice on, once you have a working agent to break.

Agent memory: what actually persists between calls

The scratchpad covered above is short-term memory — it lives only for the length of one run, and disappears the moment that run ends. **Why that's a limit worth naming:** a real assistant often needs to remember something *across* separate conversations — a user's name, a past preference, a fact established last week — and the scratchpad can't do that, because a brand new run starts with a brand new, empty message history. **The harder problem this previews:** longer-term memory means deciding what's worth saving (not everything — a full transcript of every conversation forever is neither useful nor readable), where it gets stored (a database, a file, a vector store), and how it gets back into a *future* run's context without just dumping the entire history back in and blowing past the model's context window. **When you'll actually need this:** not yet — this document's loop only needs the scratchpad — but it's worth knowing the two are genuinely different problems now, so "agent memory" doesn't get mistaken for "just keep the scratchpad around longer." Doc09 covers the real mechanism, once you have a graph to hang it on: see [Long-term memory: remembering across separate conversations](#).

Planning before acting

The ReAct loop above decides one step at a time — think, act, observe, then decide the *next* step only once the last result is in. An alternative design has the model write out a multi-step plan up front, before doing anything, and then execute that plan step by step. **The trade-off:** deciding one step at a time reacts well to surprises — a tool result the model didn't expect changes what it does next, naturally — but it can also wander, taking a longer and less predictable path to the answer. Planning first is more predictable and easier to review (you can look at the plan before any tool actually runs), but it costs more up front — an extra model call just to produce the plan — and a plan made before seeing any real results can turn out to be wrong once the first tool result comes back, needing a re-plan anyway. **When to use which:** ReAct's one-step-at-a-time default suits tasks where each step genuinely depends on the last one's result; plan-first suits tasks where the steps are more knowable in advance and predictability — or a human reviewing the plan before it runs — matters more than adapting on the fly.

When NOT to use an agent

An agent loop earns its complexity when the number and order of steps genuinely can't be known in advance — when what happens next truly depends on what the last tool returned. **When it's the wrong tool for the job:** if a task always takes the same fixed sequence of steps — always call tool A, then always tool B, then format the result — that's not a job for an agent loop deciding each step on its own; it's a plain function, or a fixed pipeline of tool calls written directly in your own code. **Why this matters:** an agent loop costs more (each step is its own model call), is slower, and is less predictable than code that just calls the same things in the same order every time — and none of that cost buys you anything if the order was never actually in question. **A simple test:** if you can write the sequence of steps down today, with confidence it won't change based on what happens along the way, write that sequence as code. Reach for an agent loop only once you can't.

Evaluating an agent, not just a single call

Checking whether an agent's *final answer* is correct isn't enough to trust it in production — an agent can land on the right answer by an expensive, fragile, or lucky path, and that path is exactly what breaks next time the input changes slightly. **Why this matters:** a wrong intermediate step that happens to get corrected later still cost real time and money, and a run like that is far more likely to fail outright on the next similar task. **What to actually check, beyond the final answer:** did it use the *right* tools for the task, rather than a wrong one that happened to still work; did it take a *reasonable* number of steps, not an inflated one; and did it avoid looping — asking for the same or a very similar action more than once without making progress. **How this looks in practice:** log every step (this document's Build Task already asks for this), and build simple checks over that log — tool-use accuracy (right tool, right arguments) and step-count sanity (it didn't take 15 steps for a 2-step task) — the same way you'd check output correctness, just one level lower, on *how* the answer was reached instead of only what it was.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail. - [ReAct: Synergizing Reasoning and Acting in Language Models \(arXiv\)](#) — the paper behind the think → act → observe loop. - [LangChain — Agents concept](#) — `create_agent`, the current standard way to get a ready-made tool-calling agent. Its older predecessor, `AgentExecutor`, is now legacy, but still worth recognizing if you read older code. - [Anthropic — Building Effective Agents](#) — read the "agents" section again now that you know tool-calling.

Practice Exercises

Setup for this document's practice code: work inside `07_ai_agents/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install`

`langchain langchain-openai` .

How to run each exercise: group your practice code by topic, not by difficulty level. If two exercises below are really about the same thing, save them together in ONE script named after that topic — for example, if two exercises are both about `.env` config, save both in one file like `env_config_practice.py` , with each level's version as its own clearly labeled section inside it. Run each topic's file directly, for example: `python react_loop_practice.py` .

For this document, save your practice code as: - **Basic** (trace the loop on paper first) and **Intermediate** (build the loop yourself) are both about the ReAct loop itself, first by hand and then in code — save them together as `react_loop_practice.py` , one section per level. - **Real-world** (compare your loop to the library's) is its own topic — save it as `agent_executor_comparison_practice.py` . - **Edge cases** (a task that needs no tool at all) is its own topic — save it as `no_tool_needed_practice.py` . - **Failure** (watch it loop, then measure the cost of looping at all) is its own topic — save it as `loop_safety_cost_practice.py` .

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to advanced, side by side.

Jump to an exercise: [Basic](#) · [Intermediate](#) · [Real-world](#) · [Edge cases](#) · [Failure](#) · [Build Task](#)

Basic — trace the loop on paper first

- **What:** trace a 3-step ReAct loop by hand on paper (prompt → thought → action → result → thought → final answer) before writing any code.
- **Why:** if you can't trace it on paper, you can't debug it in code — this is the cheapest possible way to catch a wrong mental model.
- **When you'll hit this for real:** any time an agent does something confusing and you need to reconstruct, step by step, why.
- **How to practice it:** pick a 2-tool task, and write out each thought/action/observation pair by hand, exactly as you expect the model to produce it, before touching your editor.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — build the loop yourself

- **What:** build the ReAct loop yourself (no `AgentExecutor`) for one tool.
- **Why:** this document's whole point is that you understand this loop isn't magic — building it once, yourself, is what makes that true instead of just something you read.
- **When you'll hit this for real:** Project 2's core, and every custom agent behavior you'll ever need that a prebuilt library doesn't quite support.

- **How to code it:** a `while` loop that calls the model with tools registered, checks if it requested a tool call, runs it if so and appends the result, or returns the final answer if not.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — compare your loop to the library's

- **What:** switch to `create_agent` (the current standard library function for this) with your Doc06 tools registered, and compare its behavior to your own hand-built loop.
- **Why:** now that you've built the primitive, seeing the library's version side by side tells you exactly what it's doing for you, and what it's hiding.
- **When you'll hit this for real:** reading someone else's `create_agent`-based code and needing to know what it's actually doing underneath.
- **How to code it:** wire the same tools into both your loop and `create_agent`, run the same 3 test prompts through each, and diff the final answers and step counts.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Edge cases — a task that needs no tool at all

- **What:** a prompt with no correct tool to call — check whether the agent answers directly instead of forcing a tool call anyway.
- **Why:** an agent that always reaches for a tool, even when it doesn't need one, wastes cost and time on every single call.
- **When you'll hit this for real:** the Verifier-style checks in Project 2 — a question the Worker can answer directly shouldn't trigger an unnecessary tool call.
- **How to code it:** ask something the model can answer from general knowledge alone, run it through your loop, and confirm zero tool calls happened.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure — watch it loop, then measure the cost of looping at all

- **What:** remove the step limit for a moment and watch a prompt loop (carefully, with a real budget cap in place), then put the limit back. Then measure the token/cost difference between a 5-step agent run and one direct call.
- **Why:** you need to have actually watched an agent loop forever once, so the step-limit requirement stops feeling like an abstract rule and starts feeling like something you're protecting yourself from.
- **When you'll hit this for real:** an agent given a subtly confusing tool result that makes it repeat the same action — this happens in real systems, not just exercises.
- **How to code it:** temporarily set `max_iterations=1000` (never fully unlimited) with a hard wall-clock timeout as a safety net, run an adversarial prompt, watch it loop, kill it, restore the real limit. Then compare token usage logged by the API for the looping run vs. a single direct call on an easy task.

- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Build Task — Project 2: Tool-Using Agent

Stuck on the Build Task? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Goal: a single agent with 3+ tools, real checking of arguments, and graceful handling of a tool failure mid-run.

Requirements:

- Reuses the tool library from `06_tools_function_calling`.
- A hard limit on steps — must always stop, never loop forever, even on a tricky input.
- At least one tool must be allowed to actually fail during a real run, and the agent must recover (retry, fall back, or report clearly) instead of crashing or making things up.
- Logs every step of the loop (thought/action/result) for later debugging.

Inputs: any text task typed into the terminal.

Outputs: a final answer, plus a full step-by-step log of how it got there.

Constraints: no unlimited loops, for any input. Every tool call's arguments must pass Pydantic checking before the tool actually runs.

Suggested files:

```
project_2_researchhand_tool_agent/
├─ main.py
├─ agent.py
├─ tools.py           (reused from 06_tools_function_calling)
└─ test_agent.py
```

Functions/Components to build:

- `agent.py` → `run_agent(task: str, max_iterations: int) -> AgentResult`
- a step-log recorder
- a `MaxIterationsExceeded` error, raised (not silently ignored) when the limit is hit

Expected Behavior

- Clear tasks get solved in a few steps, using the right tool(s).
- A task with no relevant tool gets a direct answer, without forcing a tool call.
- A mid-run tool failure produces a clear, logged recovery — not a crash, not a made-up success.

- Hitting the step limit raises a clear, named error, with the partial log attached.

Test Cases

Scenario	Expected
Task clearly needs Tool A	Tool A is called, correct final answer
Task needs no tool	Direct answer, zero tool calls
A tool set to fail once then succeed	The agent recovers, reaches the correct final answer
A task designed to loop forever	The step limit is hit, <code>MaxIterationsExceeded</code> raised with the log

Break-It / Debug Preview

- Missing or wrong stopping condition → an endless loop.
- The agent ignores a tool's error and states a made-up result as fact.
- Full debugging drill in [14_debugging_lab](#).

Interview Topics Preview

- ReAct vs. plain function-calling · why step limits aren't optional in production · the cost of agent loops vs. single calls.

☐ You Can Now Build Project 2

Docs 05-07 are everything Project 2 needs. Go to [project_2_researchhand_tool_agent/](#) and start with its **Setup** section — the step-by-step build guide there (not the Build Task summary above) is what you actually follow.

Move On When

Project 2 survives all the break scenarios above, without you having to guide it by hand. Full details: [CURRICULUM.md §4](#).

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full solution only if you say "**Show me the solution.**"